

Guia de replicação do experimento MQTT

Passo a passo dos comandos e arquivos. Duas máquinas Debian Linux na mesma rede: **Máquina A** = broker (Docker); **Máquina B** = gerador de carga (MQTTLoader). Cada passo indica em qual máquina executar.

Substitua o IP de exemplo

O endereço `10.0.42.71` usado neste guia é um **exemplo** e corresponde ao IP da **Máquina A**. Descubra o seu no passo 4 (`hostname -I`) e troque-o em três pontos: no teste do passo 6 (`nc`), na linha `broker =` do `piloto.conf` (passo 8) e na variável `BROKER_IP` do `run-experiment.sh` (passo 9).

1. Instalar o Docker e a imagem do broker

Máquina A

O broker Mosquitto roda em um container Docker. Instale o Docker e baixe a imagem oficial do Mosquitto:

```
sudo apt update && sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER          # adiciona seu usuario ao grupo docker
# faça logout e login (ou reinicie) para o grupo docker ter efeito
docker pull eclipse-mosquitto:2        # baixa a imagem do broker Mosquitto 2.x
```

Confirme a instalação:

```
docker --version
docker images eclipse-mosquitto        # deve listar a imagem baixada
```

2. Broker: pasta e arquivo de configuração

Máquina A

Crie a pasta de trabalho:

```
mkdir -p ~/mqtt-exp-1 && cd ~/mqtt-exp-1
```

Crie o arquivo `mosquitto.conf`. Com editor de texto:

```
nano mosquitto.conf          # digite o conteudo, salve com Ctrl+O e saia com Ctrl+X
```

Ou escrevendo direto pelo terminal, sem abrir editor:

```
cat > ~/mqtt-exp-1/mosquitto.conf << 'EOF'
listener 1883 0.0.0.0
allow_anonymous true
persistence false
EOF
```

Confira o conteúdo:

```
cat ~/mqtt-exp-1/mosquitto.conf
```

3. Broker: script de recriação Máquina A

Crie o `recreate-broker.sh`. Ele derruba o broker anterior e sobe um novo, limpo. Aceita a cota de CPU como argumento (sem argumento = sem limite):

```
cat > ~/mqtt-exp-1/recreate-broker.sh << 'EOF'
#!/usr/bin/env bash
set -euo pipefail
NAME=mosq-exp-1
CONF="$HOME/mqtt-exp-1/mosquitto.conf"
CPUS="${1:-}"
docker rm -f "$NAME" 2>/dev/null || true
CPU_FLAG=""; [ -n "$CPUS" ] && CPU_FLAG="--cpus=$CPUS"
docker run -d --rm --name "$NAME" $CPU_FLAG \
  -p 1883:1883 \
  -v "$CONF:/mosquitto/config/mosquitto.conf:ro \
  eclipse-mosquitto:2
sleep 2
echo "Broker: $NAME (CPU: ${CPUS:-sem limite})"
docker ps --filter name=$NAME --format '{{.Image}} {{.Status}}'
EOF
chmod +x ~/mqtt-exp-1/recreate-broker.sh
```

Subir o broker — sem limite e com limite de CPU

```
# SEM limite (condicao livre):
~/mqtt-exp-1/recreate-broker.sh

# COM limite de 0,02 de um nucleo (condicao restrita):
~/mqtt-exp-1/recreate-broker.sh 0.02
```

Equivalente direto pelo `docker run` (a opção `--cpus` impõe o limite):

```
docker run -d --rm --name mosq-exp-1 --cpus="0.02" \
  -p 1883:1883 \
  -v "$HOME/mqtt-exp-1/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro \
  eclipse-mosquitto:2
```

Verifique o limite aplicado (para `0.02` deve retornar `20000000`):

```
docker inspect mosq-exp-1 --format '{{.HostConfig.NanoCpus}}'
```

Se o container subir e sumir: rode em primeiro plano para ver o erro — `docker run --rm -p 1883:1883 -v "$HOME/mqtt-exp-1/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro eclipse-mosquitto:2`. Se for quebra de linha do Windows (CRLF): `sed -i 's/\r$//' ~/mqtt-exp-1/mosquitto.conf`.

4. Broker: IP e firewall Máquina A

```
hostname -I          # anote este IP; use-o nos passos 6, 8 e 9
sudo ufw status      # se "active", libere a porta:
sudo ufw allow 1883/tcp
```

5. Carga: Java e MQTTLoader Máquina B

```
sudo apt update && sudo apt install -y default-jre
java -version           # confirma o Java instalado
```

Baixe `mqttloader-0.8.6.zip` de `github.com/dist-sys/mqttloader/releases`, extraia em `~/mqttloader` e dê permissão:

```
chmod +x ~/mqttloader/bin/mqttloader
```

6. Testar a conexão com o broker Máquina B

```
nc -vz 10.0.42.71 1883    # troque pelo SEU IP da Máquina A; espera-se "succeeded"
```

7. Modo de desempenho da CPU Máquinas A e B

```
sudo apt install -y linux-cpupower
sudo cpupower frequency-set -g performance
```

8. Calibração (piloto) Máquinas A e B

Crie o `piloto.conf` na Máquina B (use o IP real na primeira linha):

```
cat > ~/mqtt-exp-1/piloto.conf << 'EOF'
broker = 10.0.42.71           # <-- troque pelo SEU IP da Máquina A
broker_port = 1883
mqtt_version = 5
num_publishers = 50
num_subscribers = 1
qos_publisher = 1
qos_subscriber = 1
topic = exp/throughput
payload = 64
interval = 100000
num_messages = 1000000
exec_time = 30
ramp_up = 0
ramp_down = 0
subscriber_timeout = 10
log_level = INFO
EOF
```

interval é em microssegundos. Para 100 ms use `100000`. Valores menores (ex.: `1000`) viram teste de estresse.

Na Máquina A recrie o broker com a cota; na Máquina B dispare o piloto:

```
# Máquina A:  
~/mqtt-exp-1/recreate-broker.sh 0.02  
# Máquina B:  
~/mqttlloader/bin/mqttlloader -c ~/mqtt-exp-1/piloto.conf
```

No bloco `-----Subscriber-----`, a cota está boa quando as mensagens recebidas ficam **abaixo** das publicadas e a latência média sobe muito com 50 publishers, mas permanecem normais com `num_publishers = 10`. Neste experimento usou-se `0.02`.

9. Coleta automatizada Máquina B

Crie o orquestrador. Ele percorre os cenários e as repetições, recria o broker entre rodadas e salva cada CSV como `pubN_runR.csv`:

```

cat > ~/mqtt-exp-1/run-experiment.sh << 'EOF'
#!/usr/bin/env bash
set -uo pipefail

# ===== EDITE AQUI =====
SCENARIOS=(1 5 10 20 30 50)
REPETITIONS=2
EXEC_TIME=60
INTERVAL_US=100000
PAYLOAD=64
QOS=1
TOPIC="exp/throughput"
MQTT_VERSION=5
RAMP_UP=3
RAMP_DOWN=3

CONDICA0="restrito"          # "livre" ou "restrito"
CPUS_RESTRITO="0.02"

BROKER_IP="10.0.42.71"      # <-- troque pelo SEU IP da Maquina A
BROKER_PORT=1883

MQTTLOADER_DIR="$HOME/mqttloader"
WORK_DIR="$HOME/mqtt-exp-1"

USE_SSH=false
SSH_USER="aluno"
# =====

if [ "$CONDICA0" = "restrito" ]; then CPUS="$CPUS_RESTRITO"; else CPUS=""; fi
RAW_DIR="$WORK_DIR/raw"
RESULTS_DIR="$WORK_DIR/results-$CONDICA0"
LAUNCHER="$MQTTLOADER_DIR/bin/mqttloader"
CONF_RUN="$WORK_DIR/run.conf"
mkdir -p "$RAW_DIR" "$RESULTS_DIR"
[ -x "$LAUNCHER" ] || { echo "ERRO: $LAUNCHER nao executavel"; exit 1; }

recreate_broker () {
  if [ "$USE_SSH" = true ]; then
    ssh "$SSH_USER@$BROKER_IP" "bash ~/mqtt-exp-1/recreate-broker.sh $CPUS"; sleep 3
  else
    read -rp " >> Na Maquina A rode: ./recreate-broker.sh $CPUS e tecle ENTER "
  fi
}

for pub in "${SCENARIOS[@]"; do
  for run in $(seq 1 "$REPETITIONS"); do
    echo "=== Publishers=$pub Execucao=$run/$REPETITIONS [$CONDICA0] ==="
    recreate_broker
    rm -f "$RAW_DIR"/*.csv 2>/dev/null
    cat > "$CONF_RUN" <&1 | tee "$RESULTS_DIR/pub${pub}_run${run}.console.txt"
    csv=$(find "$RAW_DIR" -maxdepth 1 -name '*.csv' -printf '%T@ %p\n' 2>/dev/null \
      | sort -rn | head -1 | cut -d' ' -f2-)
    [ -n "$csv" ] && mv "$csv" "$RESULTS_DIR/pub${pub}_run${run}.csv"
    sleep 3
  done
done
echo "Concluido [$CONDICA0]. CSVs em $RESULTS_DIR"

```

```
EOF
chmod +x ~/mqtt-exp-1/run-experiment.sh
```

Rode duas vezes, trocando a variável `CONDICAO` no topo do script:

```
# CONDICAO="livre"      -> ./run-experiment.sh    (resultados em results-livre/)
# CONDICAO="restrito"   -> ./run-experiment.sh    (resultados em results-restrito/)
~/mqtt-exp-1/run-experiment.sh
```

10. Análise

```
pip install numpy matplotlib
```

Análise de cada condição (edite `RESULTS_DIR` no topo do script para cada pasta):

```
python3 analise_mqtt.py
```

Comparação entre as duas condições (gera tabela e figuras sobrepostas):

```
python3 comparador_mqtt.py
```

Janelas de descarte: `RAMP_UP_S` e `RAMP_DOWN_S` nos scripts de análise devem ser iguais aos da coleta (3 e 3).

11. Problemas comuns

- Porta 1883 recusa conexão externa → falta `listener 1883 0.0.0.0` no `mosquitto.conf`, ou firewall: `sudo ufw allow 1883/tcp`.
- Broker sobe e some → rode em primeiro plano para ver o erro; se CRLF: `sed -i 's/\r$//'` `mosquitto.conf`.
- Throughput em milhares de msg/s → `interval` ausente ou errado; confirme `interval = 100000`.
- Latência menor com mais publishers → governador de energia; rode `cpupower frequency-set -g performance` nas duas máquinas.
- Análise diz “sem dados” → caminho configurado diferente da pasta real; alinhe `RESULTS_DIR` / `DIR_*`.